UNIVERSITY OF SCIENCE
VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
Faculty of Information Technology

Course: Introduction to Software Engineering

SOFTWARE ARCHITECTURE DOCUMENT

Workflow Management System

Subject:	Software Architecture and Class Design
Topic:	Workflow Management System
Full name:	Huỳnh Tấn Phước
Student ID:	23120334
Instructor:	Thạc sĩ Phạm Hoàng Hải

Ho Chi Minh City, June 3, 2026

Contents

1 Introduction

1.1 Purpose

This Software Architecture Document (SAD) describes the architecture and design of the **Workflow Management System**.

Document focus

- Overall system architecture
- Main system components
- Relationships between components
- Workflow management functionalities
- Class design and responsibilities

The purpose of this document is to support system implementation, maintenance, and future extension.

1.2 Scope

The Workflow Management System is part of a virtual company operation platform using AI-agents.

Supported functions

- Workflow creation and editing
- Workflow validation
- Workflow execution
- Execution monitoring
- Execution log management
- Agent management

This document mainly focuses on the software architecture and class design of the Workflow Management feature.

1.3 References

- Vision Document
- Use-case Specification Document
- Project Assignment 3 (PA3) Requirements

2 Architectural Goals and Constraints

2.1 Architectural Goals

- Separate presentation, business logic, and data access responsibilities
- Support workflow orchestration between multiple AI-agents
- Improve maintainability and modularity
- Support future scalability and extension
- Provide execution monitoring and logging capabilities

2.2 Architectural Constraints

- The system follows the N-tier architectural style
- The application is designed as a web-based platform
- Workflow execution must support multiple agents
- The architecture must remain consistent with the use-case model
- The project is developed within the academic scope of CS300

3 Use-Case Model

3.1 Overview

The following use-case diagram describes the main functionalities of the Workflow Management System and interactions between actors and the system.

Main actors

- Workflow Creator
- Agent Supervisor
- Workflow Executor

Main use cases

- Create Workflow
- Edit Workflow
- Validate Workflow
- View Workflow List
- Execute Workflow
- Monitor Execution Status
- View Execution Logs
- Manage Agents

3.2 Use-Case Diagram

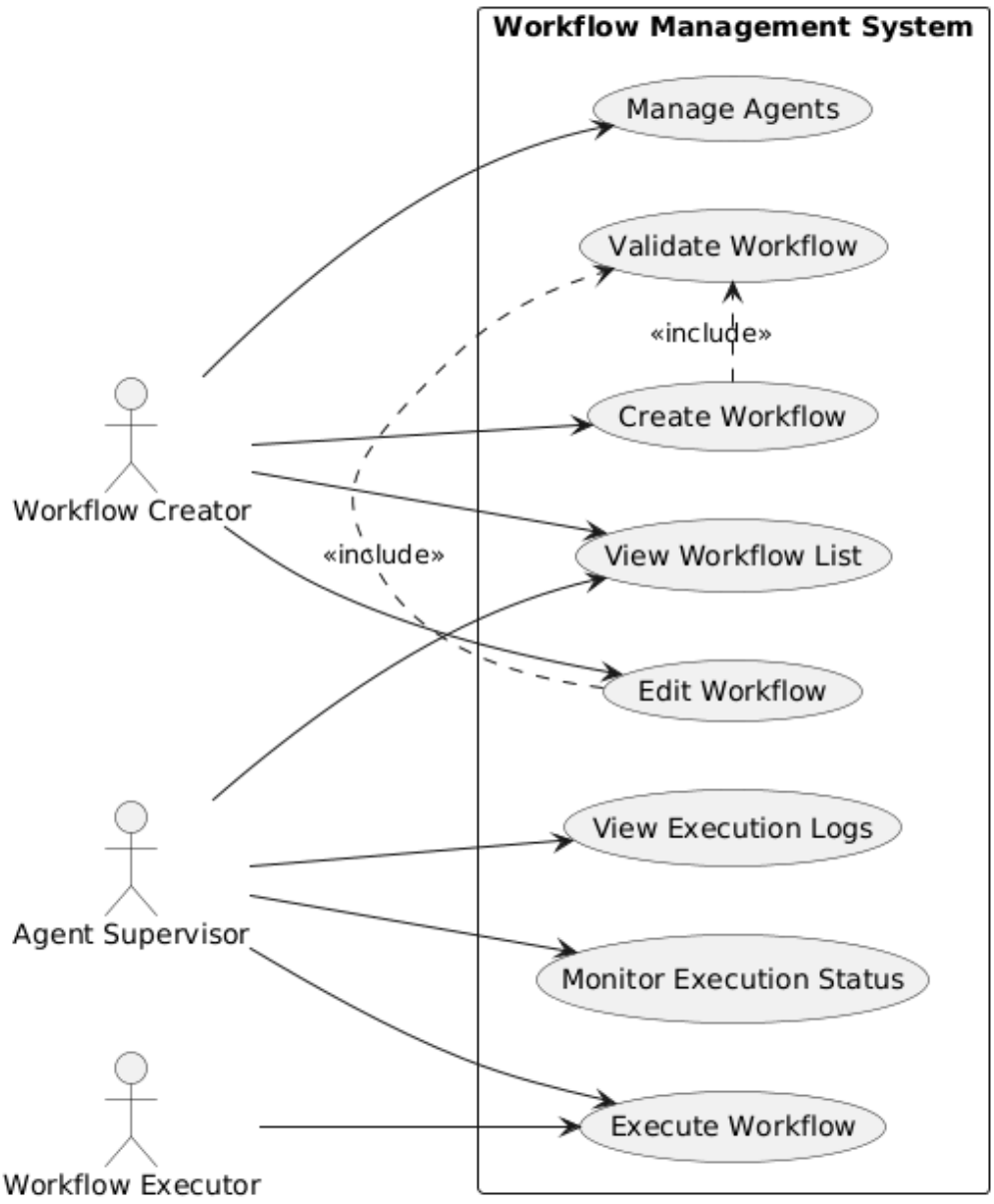

Figure 1: Use-case diagram of the Workflow Management System

4 Logical View

4.1 Purpose of the Logical View

The Logical View explains how the Workflow Management System is decomposed into layers, components, and classes. It focuses on the internal organization of the system rather than physical deployment details.

The design follows a clear top-down structure:

- Users interact with the system through presentation components.
- Business services process workflow requests and coordinate validation or execution.
- Repository components isolate data access logic.
- The database stores workflows, steps, executions, agents, and logs.

4.2 Layered Architecture

The system uses an N-tier structure to separate user interaction, business processing, data access, and persistent storage. This separation makes the system easier to maintain, test, and extend.

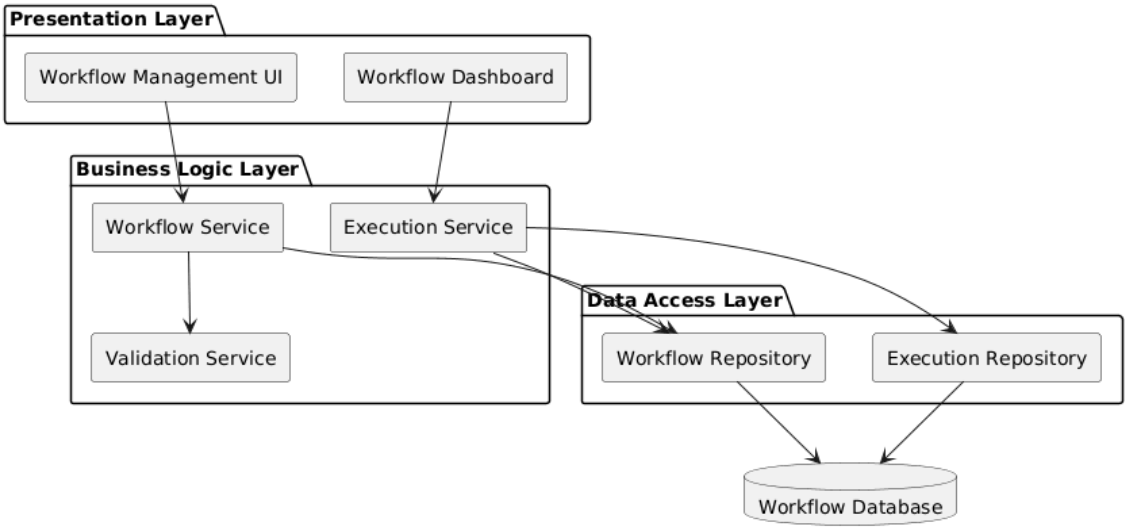

Figure 2: Architecture diagram of the Workflow Management System

Layer responsibilities

Layer	Main Components	Responsibility
Presentation Layer	Workflow Management UI; Workflow Dashboard	Provides user-facing screens for creating workflows, managing agents, monitoring execution, and viewing logs.
Business Logic Layer	Workflow Service; Validation Service; Execution Service	Applies workflow rules, validates workflow structure, coordinates execution, and manages execution status.
Data Access Layer	Workflow Repository; Execution Repository	Encapsulates database operations and provides a stable interface for services to access stored data.
Database Layer	Workflow Database	Stores workflow definitions, workflow steps, execution records, execution logs, and agent-related data.

4.3 Component Design

This subsection describes the responsibilities and communication paths of the main components. The components are grouped by architectural layer so that their roles are easier to understand.

4.3.1 Presentation Layer Components

Component	Responsibilities	Communication
Workflow Management UI	Allows workflow creators to create workflows, edit workflows, view workflow lists, and manage agents.	Sends workflow management requests to Workflow Service through HTTP/HTTPS.

Workflow Dashboard	Allows users to monitor workflow execution, view execution progress, and inspect execution logs.	Sends monitoring and log requests to Execution Service through HTTP/HTTPS.

4.3.2 Business Logic Layer Components

Component	Responsibilities	Communication
Workflow Service	Handles workflow creation, editing, retrieval, and validation coordination.	Receives requests from Workflow Management UI; uses Validation Service and Workflow Repository.
Validation Service	Checks workflow structure, step dependencies, and assigned agents before a workflow is saved or executed.	Receives validation requests from Workflow Service.
Execution Service	Executes workflows, monitors execution status, and generates execution logs.	Receives requests from Workflow Dashboard; uses Workflow Repository and Execution Repository.

4.3.3 Data Access and Database Components

Component	Responsibilities	Communication
Workflow Repository	Saves, updates, and retrieves workflow definitions and workflow steps.	Used by Workflow Service and Execution Service; accesses Workflow Database.
Execution Repository	Saves execution records, updates execution status, and retrieves execution history or logs.	Used by Execution Service; accesses Workflow Database.
Workflow Database	Persists workflows, workflow steps, executions, execution logs, and agent data.	Accessed only through repository components.

4.4 Main Processing Flow

The logical flow of the system can be summarized as follows:

1. A user creates or edits a workflow through the Workflow Management UI.
2. The Workflow Management UI sends the request to Workflow Service.
3. Workflow Service calls Validation Service to check workflow correctness.
4. If the workflow is valid, Workflow Service stores it through Workflow Repository.
5. When execution is requested, Execution Service retrieves workflow data from Workflow Repository.
6. Execution Service coordinates workflow execution and stores execution results through Execution Repository.
7. Workflow Dashboard displays execution status and logs to the user.

4.5 Class Design

The class design supports the layered architecture by separating domain entities, business services, and repositories. Domain classes represent the data model, service classes coordinate business operations, and repository classes handle persistence.

4.5.1 Class Diagram

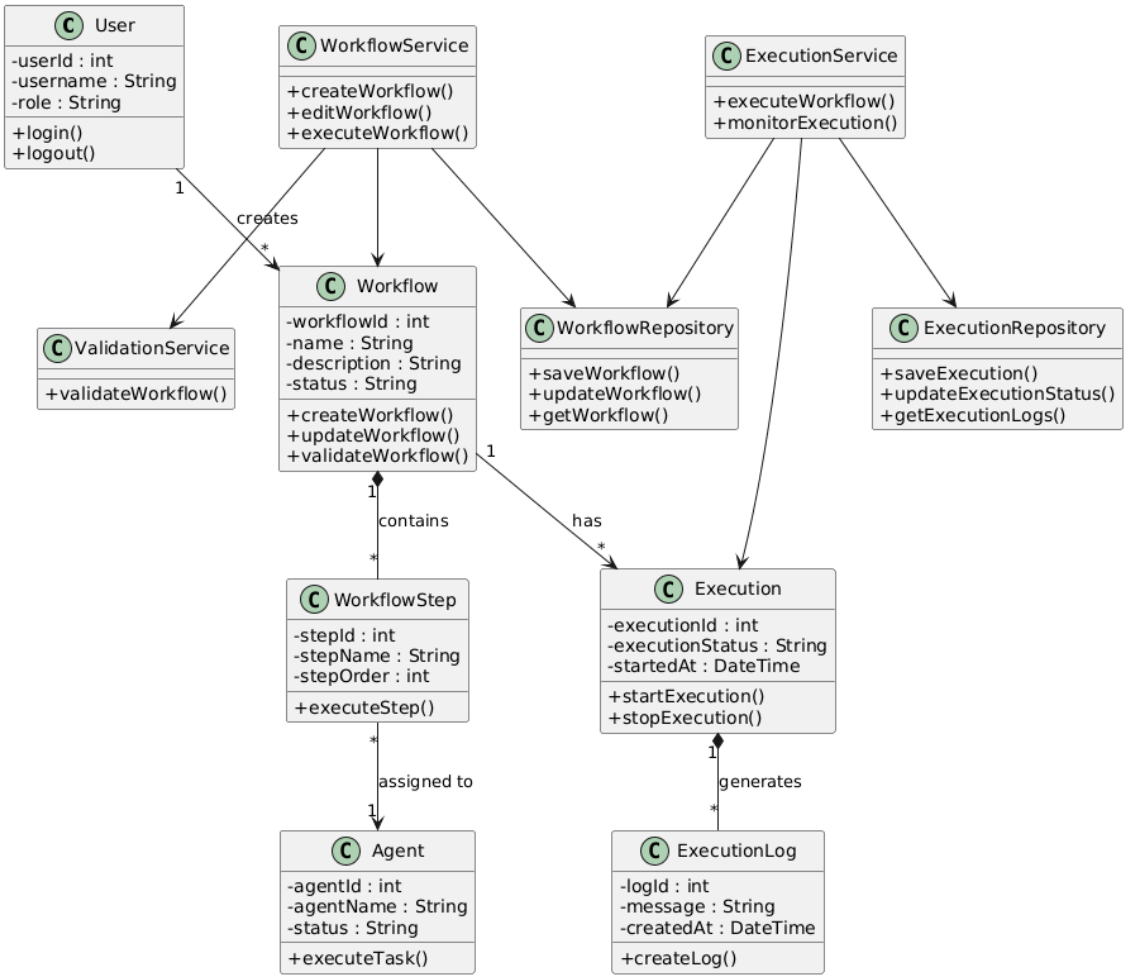

Figure 3: Class diagram of the Workflow Management System

4.5.2 Class Groups

Group	Classes	Purpose
Workflow Entity Classes	User; Workflow; WorkflowStep; Agent	Model users, workflow definitions, workflow steps, and assigned agents.
Execution Entity Classes	Execution; ExecutionLog	Track workflow execution instances, execution status, and generated logs.
Service Classes	WorkflowService; ValidationService; ExecutionService	Coordinate business logic for workflow management, validation, execution, and monitoring.
Repository Classes	WorkflowRepository; ExecutionRepository	Provide data access operations for workflow data and execution data.

4.6 Logical View Summary

The Logical View shows that the Workflow Management System is organized around a clear separation of concerns. Presentation components handle user interaction, services implement business rules, repositories manage data access, and entity classes represent workflow and execution information. This structure supports maintainability, scalability, and future extension.

5 Deployment

5.1 Deployment Overview

This section describes how the current PA4 Workflow Management System prototype is deployed. The current implementation is a web-based frontend prototype built with React, TypeScript, Vite, and Tailwind CSS. Users access the system through a web browser. The application is served by the Vite development server during development or by a static web server after the project is built.

In the current PA4 scope, the system does not include an implemented backend server, REST API layer, database server, or external agent runtime. Workflow, execution, and agent information shown in the interface are represented by static or in-memory data inside React components. Therefore, the current deployment focuses on the client browser and frontend application server.

For future extensions, the system can be expanded with a backend application server, persistent database, and optional agent runtime services. These future nodes would support real workflow persistence, workflow execution, validation, monitoring, and agent-based task execution.

5.2 Deployment Nodes

Node	Deployed Components	Responsibility
Client Device	Web Browser	Provides access to the Workflow Manager user interface, including Dashboard, Workflow Management, Workflow Editor, Executions, Agents, and Settings screens.
Frontend Development / Static Web Server	Vite application bundle; React components; CSS files; UI assets	Serves the frontend application to the browser. During development, this role is handled by the Vite development server. After build, it can be handled by any static web server.
Backend Application Server	Not implemented in current PA4; planned for future extension	Would provide REST APIs for workflow management, validation, execution, monitoring, and agent management.
Database Server	Not implemented in current PA4; planned for future extension	Would store workflow definitions, workflow steps, agents, execution records, statuses, and logs.
Optional Agent Runtime Node	Not implemented in current PA4; planned for future extension	Would execute workflow steps assigned to agents if the system is extended beyond the frontend prototype.

5.3 Communication Between Nodes

Communication Path	Protocol	Description
Client Device → Frontend Development / Static Web Server	HTTP / HTTPS	The browser loads the React application, JavaScript bundle, CSS files, and static assets.
Browser → React Application State	Internal frontend state	Page navigation and UI interactions are handled inside the frontend using React state.
React Application → Backend Application Server	Not implemented	In the current prototype, no REST API calls are implemented. This path is reserved for future backend integration.

Backend Application Server → Database Server	Not implemented	Persistent database communication is not part of the current PA4 implementation.

5.4 Component-to-Node Mapping

Application Component	Deployment Node	Notes
Workflow Manager UI	Client Device / Frontend Server	Displayed in the browser and served as frontend assets.
Dashboard Page	Client Device / Frontend Server	Shows workflow statistics, recent workflows, recent activity, and quick actions.
Workflow Management Page	Client Device / Frontend Server	Shows workflow list, search input, summary cards, and workflow actions.
Workflow Editor Page	Client Device / Frontend Server	Allows users to edit workflow name, description, steps, assigned agents, dependencies, and validation status.
Executions Page	Client Device / Frontend Server	Shows execution statistics and execution history using prototype data.
Agents Page	Client Device / Frontend Server	Shows available agents, agent types, statuses, and active workflow counts.
Settings Page	Client Device / Frontend Server	Shows notification and auto-save settings in the prototype interface.
Workflow Service / Repository	Future Backend Server	Not implemented in current PA4. Planned for future persistence and business logic.
Workflow Database	Future Database Server	Not implemented in current PA4. Planned for persistent workflow and execution data.
Agent Runtime	Future Agent Runtime Node	Not implemented in current PA4. Planned for real agent-based execution.

5.5 Deployment Summary

The current deployment design is simple and appropriate for the PA4 prototype. The system is deployed as a frontend-only web application served through Vite or a static web server. Since the current implementation focuses on the user interface, backend services, database storage, and agent runtime execution are treated as future extensions rather than implemented deployment nodes.

6 Implementation View

6.1 Implementation Overview

This section describes the current source code organization of the PA4 Workflow Management System prototype. The implementation is organized as a React and TypeScript frontend application generated from a Figma-based design bundle. It uses Vite as the development and build tool, Tailwind CSS for styling, lucide-react for icons, and reusable UI components.

Unlike the earlier proposed structure, the current PA4 repository does not contain separate `frontend/`, `backend/`, or `docs/` folders. The implemented application is located mainly under `src/app/`. Page navigation is handled inside `App.tsx` using React state instead of a separate route configuration file.

6.2 Current Project Folder Structure

Listing 1: Current folder structure of the PA4 Workflow Management System prototype

```
PA4/
|
|-- index.html
|-- package.json
|-- vite.config.ts
|-- README.md
|
|-- src/
|   |-- main.tsx
|   |
|   |-- app/
|   |   |-- App.tsx
|   |   |
|   |   |-- components/
|   |       |-- Sidebar.tsx
|   |       |-- PageHeader.tsx
|   |       |-- StatCard.tsx
|   |       |-- StatusBadge.tsx
|   |       |
|   |       |-- pages/
|   |       |   |-- Dashboard.tsx
|   |       |   |-- Workflows.tsx
|   |       |   |-- WorkflowEditor.tsx
|   |       |   |-- Executions.tsx
|   |       |   |-- Agents.tsx
|   |       |   |-- Settings.tsx
|   |       |
|   |       |-- ui/
|   |       |   |-- button.tsx
|   |       |   |-- card.tsx
|   |       |   |-- table.tsx
|   |       |   |-- dialog.tsx
|   |       |   |-- input.tsx
|   |       |   |-- select.tsx
|   |       |   |-- tabs.tsx
|   |       |   |-- tooltip.tsx
|   |       |   |-- sonner.tsx
|   |       |   |-- ...
|   |       |
|   |       |-- figma/
|   |           |-- ImageWithFallback.tsx
|   |
|   |-- styles/
|       |-- index.css
|       |-- globals.css
|       |-- theme.css
|       |-- tailwind.css
|       |-- fonts.css
|
|-- guidelines/
|   |-- Guidelines.md
```

6.3 Frontend Structure

The frontend is organized around reusable layout components, common display components, page components, UI primitives, and style files.

Folder / File	Purpose
src/main.tsx	Entry point that renders the React application into the root HTML element.
src/app/App.tsx	Main application component. It stores the current page state and renders the selected page.
components/Sidebar.tsx	Provides the main sidebar navigation between Dashboard, Workflows, Workflow Editor, Executions, Agents, and Settings.
components/PageHeader.tsx	Reusable page heading component for page title and subtitle.
components/StatCard.tsx	Reusable statistic card used in Dashboard, Workflows, and Executions pages.
components/StatusBadge.tsx	Reusable status label for workflow and execution states such as Running, Completed, Failed, and Draft.
components/pages/Dashboard.tsx	Displays workflow statistics, recent workflows, recent activity, and quick action buttons.
components/pages/Workflows.tsx	Displays workflow list, search input, summary cards, and actions such as Create, Edit, and Execute.
components/pages/WorkflowEditor.tsx	Provides workflow editing fields, step table, assigned agents, dependencies, validation checks, and save/publish actions.
components/pages/Executions.tsx	Displays execution statistics and a table of workflow execution history.
components/pages/Agents.tsx	Displays agent list, agent types, statuses, and number of active workflows.
components/pages/Settings.tsx	Displays general settings such as notifications and auto-save drafts.
components/ui/	Contains reusable UI primitives and generated shadcn/Radix-style components.
src/styles/	Contains global CSS, theme styles, Tailwind styles, and font definitions.

6.4 Backend Structure

The current PA4 repository does not include an implemented backend structure. There are no backend controllers, services, repositories, entities, DTOs, database configuration files, or API wrapper services in the current codebase.

If the system is extended in the future, a backend structure may be added to support real workflow storage, validation, execution, and monitoring. A possible future backend organization is shown below.

Listing 2: Possible future backend folder structure

```
backend/
|-- src/
|   |-- modules/
|   |   |-- workflow/
|   |   |   |-- workflow.controller.ts
|   |   |   |-- workflow.service.ts
|   |   |   |-- workflow.repository.ts
|   |   |   |-- workflow.entity.ts
|   |   |   |-- workflow-step.entity.ts
|   |   |   |
|   |   |-- validation/
|   |   |   |-- validation.service.ts
|   |   |   |
|   |   |-- execution/
|   |   |   |-- execution.controller.ts
|   |   |   |-- execution.service.ts
|   |   |   |-- execution.repository.ts
|   |   |   |-- execution.entity.ts
|   |   |   |-- execution-log.entity.ts
|   |   |   |
```

```
|   |   |-- agent/
|   |       |-- agent.controller.ts
|   |       |-- agent.service.ts
|   |       |-- agent.repository.ts
|   |       |-- agent.entity.ts
|   |
|   |-- config/
|   |   |-- database.config.ts
|   |   |-- app.config.ts
|   |
|   |-- main.ts
```

6.5 Component-to-Folder Mapping

Application Component	Current Frontend Location	Backend Location
Main Application Shell	`src/app/App.tsx`; `components/Sidebar.tsx`	Not implemented
Dashboard	`components/pages/Dashboard.tsx`; `StatCard.tsx`; `StatusBadge.tsx`	Not implemented
Workflow Management	`components/pages/Workflows.tsx`; `StatusBadge.tsx`	Future `modules/workflow/`
Workflow Editor	`components/pages/WorkflowEditor.tsx`	Future `modules/workflow/` and `modules/validation/`
Execution Monitoring	`components/pages/Executions.tsx`; `StatCard.tsx`; `StatusBadge.tsx`	Future `modules/execution/`
Agent Management	`components/pages/Agents.tsx`	Future `modules/agent/`
Settings	`components/pages/Settings.tsx`	Future `config/`
Reusable UI Components	`components/ui/`; `PageHeader.tsx`; `StatCard.tsx`; `StatusBadge.tsx`	Not applicable

6.6 UI Prototype Screens

The following UI prototype screens are implemented in the current PA4 frontend. These screens represent the main workflow management functions and are used as the basis for future full-system implementation.

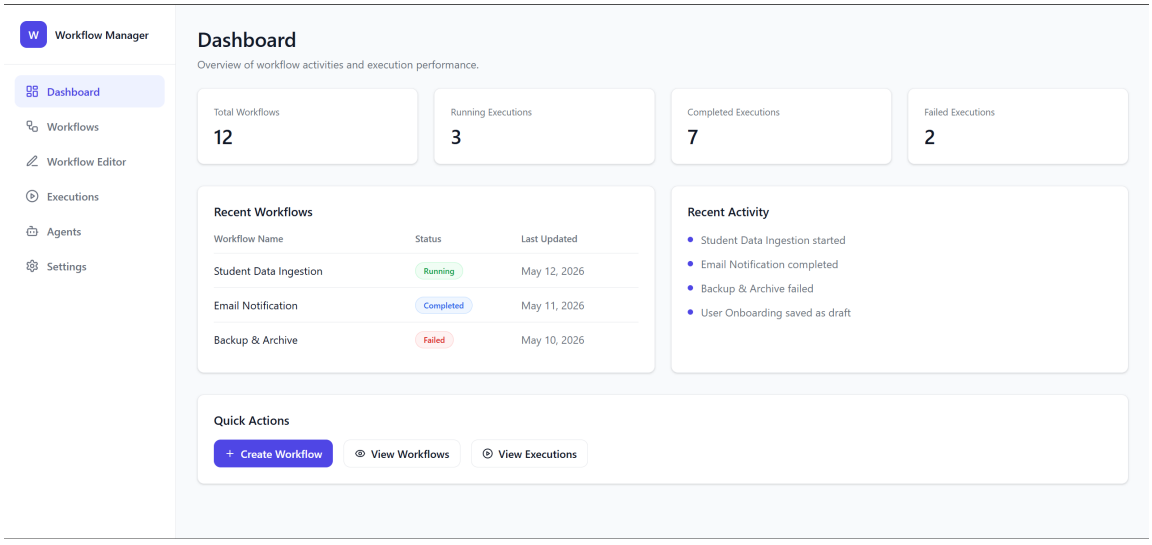

Figure 4: Dashboard screen

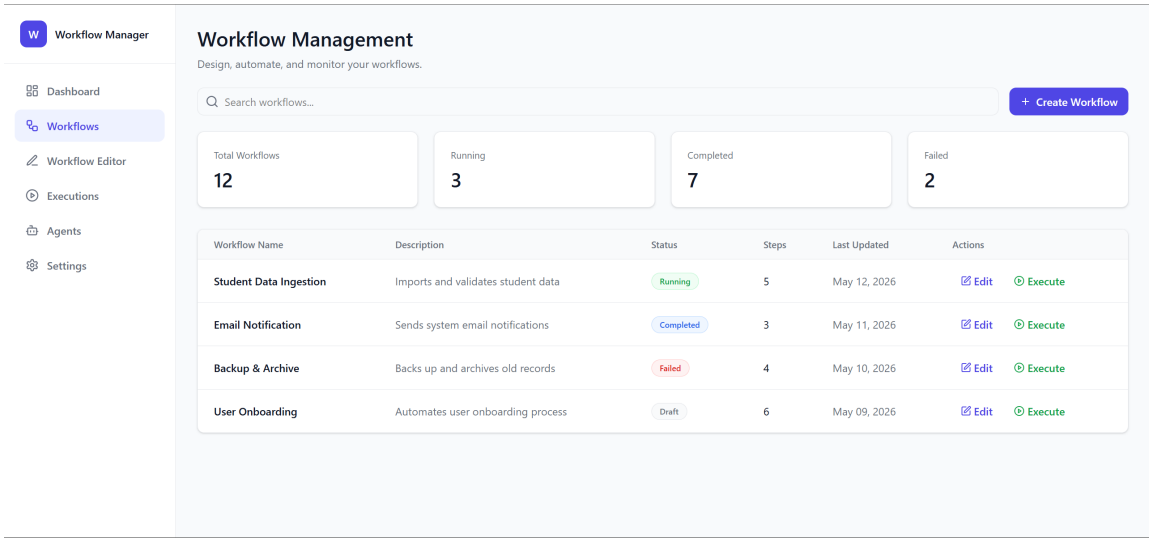

Figure 5: Workflow Management screen

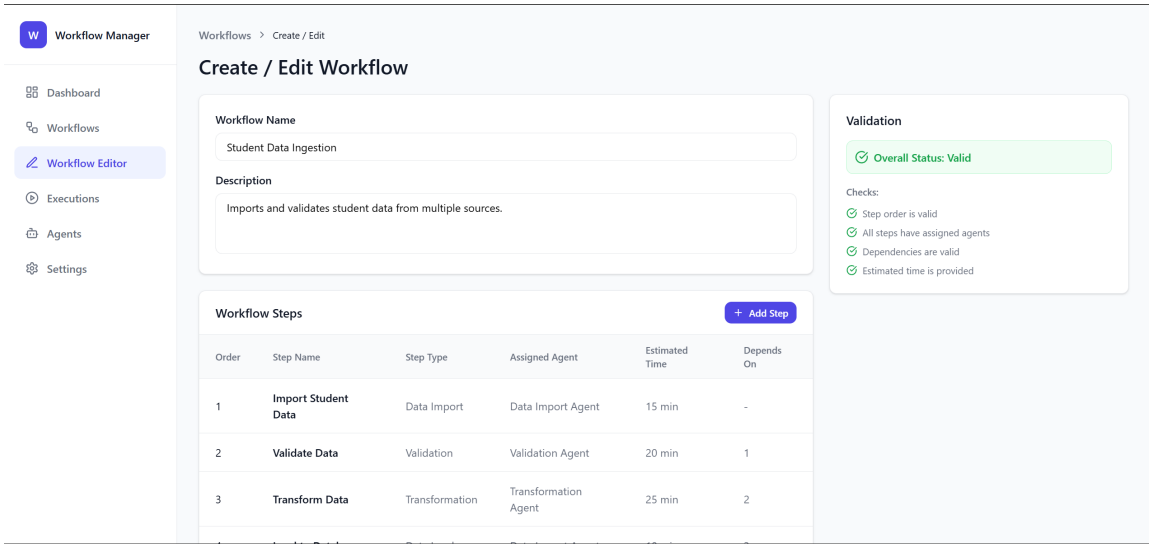

Figure 6: Create / Edit Workflow screen

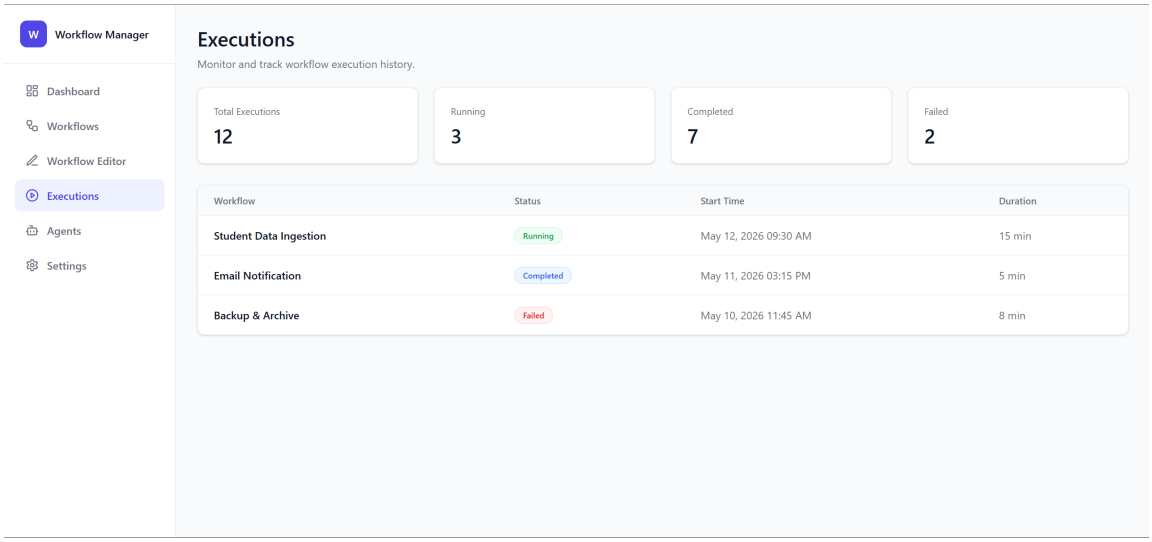

Figure 7: Execution Monitoring screen

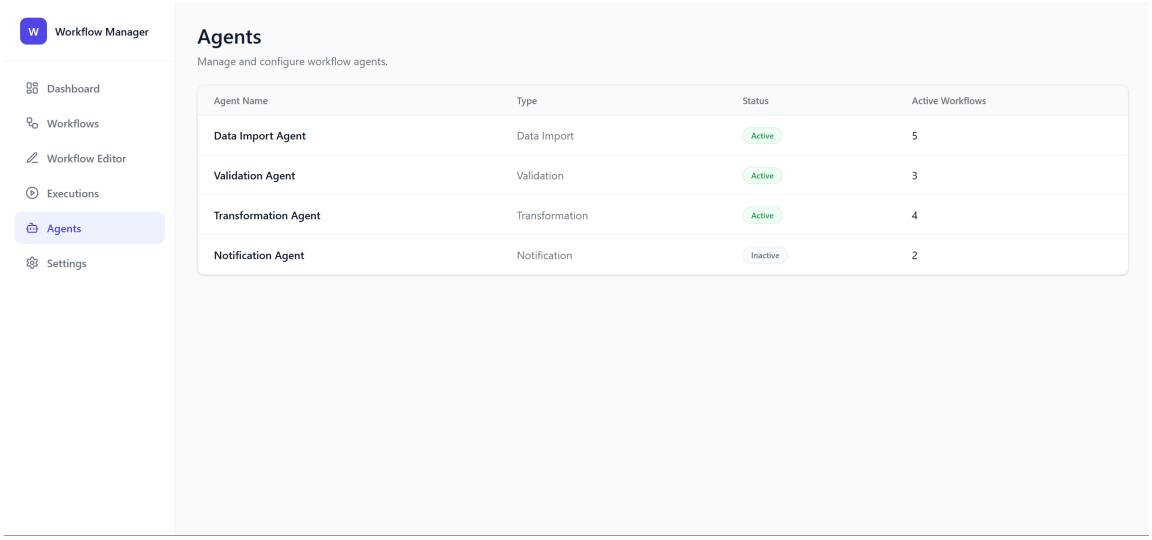

Figure 8: Agent Management screen

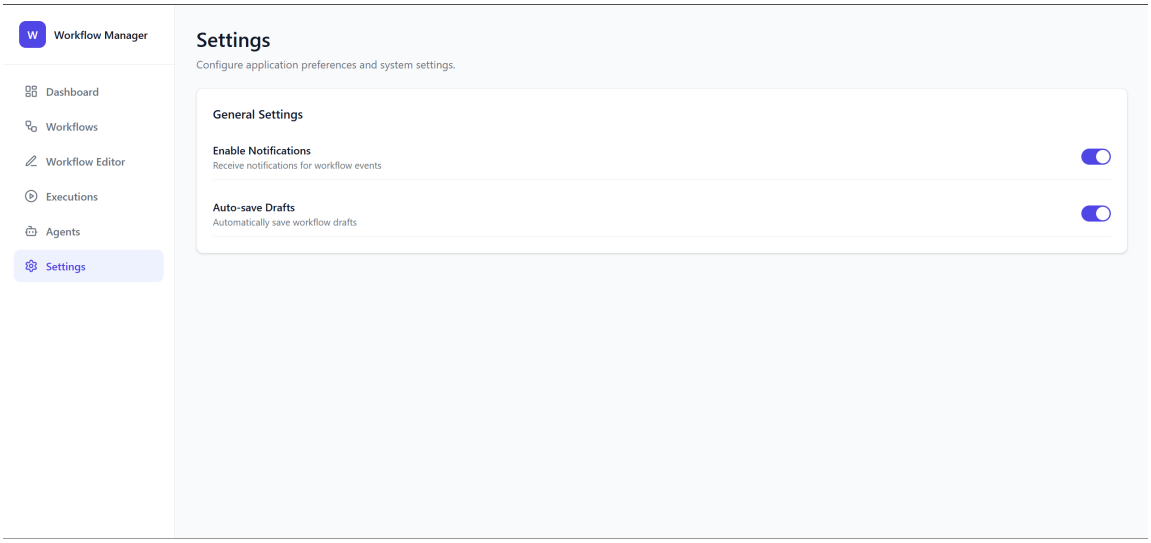

Figure 9: Settings screen

6.7 Screen Descriptions

Screen	Purpose	Main Information and Interaction
Dashboard	Provides an overview of workflow activity.	Shows total workflows, running executions, completed executions, failed executions, recent workflows, recent activity, and quick action buttons.
Workflow Management	Allows users to manage workflow definitions.	Shows workflow list, search input, status summary cards, and actions such as Create, Edit, and Execute.
Create / Edit Workflow	Allows users to define or modify workflows.	Shows workflow name, description, workflow steps, step type, assigned agent, estimated time, dependencies, validation checks, and actions such as Validate, Save Draft, and Publish.
Execution Monitoring	Allows users to monitor workflow execution history.	Shows execution summary statistics and execution records with workflow name, status, start time, and duration.
Agent Management	Allows users to view workflow agents.	Shows available agents, agent type, status, and number of active workflows.
Settings	Allows users to configure application preferences.	Shows notification and auto-save draft options.

6.8 Implementation View Summary

The Implementation View reflects the current PA4 codebase as a frontend-only Workflow Management System prototype. The implemented structure is centered on React page components, shared UI components, sidebar navigation, and global styling. Backend services, REST APIs, repositories, database storage, and external agent runtime services are not yet implemented and should be described as future extensions rather than current implementation details.